

Analyse de LunarLander

Alex Maloigne et Kamel Nehlil

1 Présentation de LunarLander

Lunar Lander est un environnement d'apprentissage par renforcement de Gym, simulant l'atterrissage d'un module spatial sur une surface lunaire à l'aide du moteur physique Box2D. L'agent doit contrôler ses moteurs afin d'atteindre une zone d'atterrissage tout en minimisant sa vitesse et sa consommation de carburant. L'espace d'observation est composé de 8 variables, incluant la position, la vitesse et l'orientation du lander. L'environnement propose un espace d'actions discret, comprenant 4 actions possibles : ne rien faire, activer le moteur principal, activer le moteur latéral gauche ou activer le moteur latéral droit. La fonction de récompense encourage un atterrissage précis et stable, tout en pénalisant les crashes et l'utilisation excessive de carburant. Cet environnement sera utilisé pour tester l'algorithme DQN dans un cadre d'apprentissage avec des observations partielles.

2 Analyse des wrappers basiques

Nous avons 4 courbes :

- En orange, DQN sans x point, y point et theta point.
- En rouge, DQN sans x point, y point et theta point mais avec en mémoire la valeur précédente de théta.
- En rose, DQN sans x point, y point et theta point mais avec en mémoire la valeur précédente de x et y.
- En gris, DQN sans x point, y point et theta point mais avec en mémoire la valeur précédente de x,y et théta.

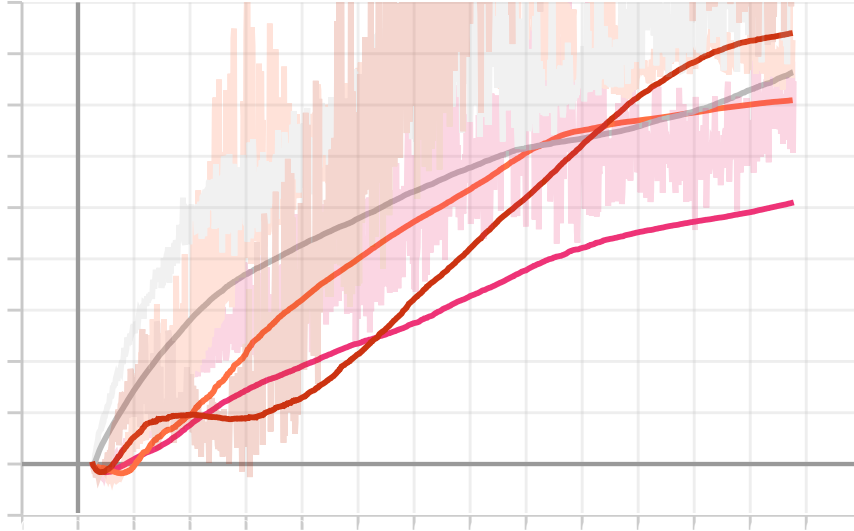


Figure 1: Valeur moyenne des q valeurs

On observe que les Q-values augmentent plus rapidement lorsque l'agent dispose d'une mémoire lui permettant d'estimer les observations manquantes, par rapport à un apprentissage sans mémoire. Dans Lunar Lander, l'objectif repose principalement sur le contrôle de l'orientation et de la vitesse d'atterrissage, plutôt que sur la position absolue. Ainsi, les paramètres liés à l'angle et à la vitesse verticale sont plus utiles en mémoire que la simple position spatiale du lander. Les Q-values montrent alors une meilleure convergence et une plus grande facilité à maximiser la récompense lorsque ces informations sont prises en compte. L'observation de ces paramètres clés permet de mieux résoudre le problème du monde partiellement observable, car l'agent peut ainsi estimer les variables manquantes et adopter une stratégie d'atterrissage plus stable et efficace.

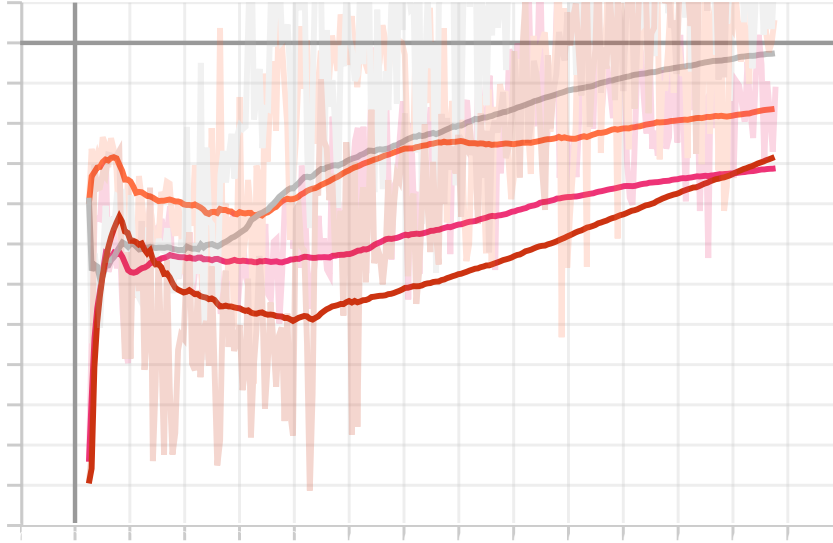


Figure 2: Valeur moyenne des rewards

Comme le montre le graphique ci-dessus, les résultats varient considérablement en fonction de la mémoire utilisée. Initialement, la courbe sans mémoire reste relativement constante après avoir atteint un certain seuil de récompense. L'ajout de la mémoire des positions (x, y) entraîne également une stagnation après un certain temps. En revanche, l'ajout de la mémoire θ entraîne une augmentation continue, offrant une pente plus marquée vers la fin, bien qu'elle ne dépasse pas les deux autres courbes. On peut supposer que si l'algorithme continue de fonctionner plus longtemps, cette courbe finirait par surpasser les autres. Enfin, l'agent utilisant les deux mémoires bat tous les records observés et continue de croître.

3 Analyse de Lunar Lander avec l'action Wrapper

Dans cette partie, nous allons expérimenter avec l'Action Wrapper et ses effets potentiellement positifs. Nous testerons 4 scénarios différents dans l'environnement partiellement observable défini plus haut (sans x, y et sans θ):

- Sans mémoire
- Avec mémoire des positions (x, y)
- Avec mémoire sur θ
- Avec mémoire combinée des positions et de θ

Le problème lié à l'Action Wrapper a été de trouver les bons paramètres à lui associer. En effet, nous avons remarqué qu'en augmentant la taille des actions prédites, les résultats se dégradaient, probablement à cause d'une surcharge d'informations ou d'une complexité accrue dans les prédictions. Ainsi, après plusieurs essais, nous avons déterminé que 2 à 3 actions constituaient le paramètre optimal pour l'Action Wrapper, offrant un bon compromis entre performance et stabilité de l'algorithme.

3.1 Sans mémoire

Ce cas est sûrement le moins intéressant à analyser pour autant nous avons d'assez bons résultats dessus.

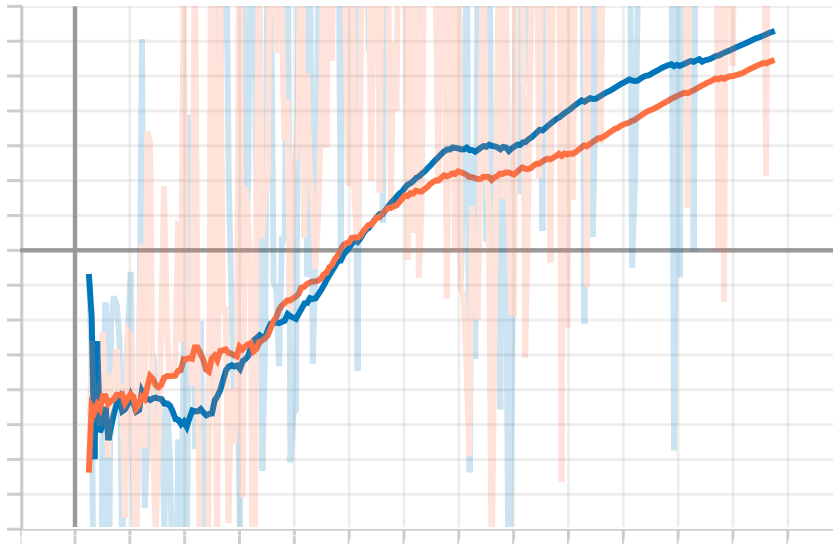


Figure 3: Valeur moyenne des rewards

Comme nous pouvons le voir, l'Action Wrapper augmente les performances de manière assez significative de l'ordre de 50 points de récompenses. Cette amélioration est clairement visible et montre l'impact positif de l'Action Wrapper sur les résultats de l'algorithme.

3.2 Avec mémoire des positions (x,y) ou sur théta

Les cas suivants montrent bien plus significativement l'impact de l'action wrapper :

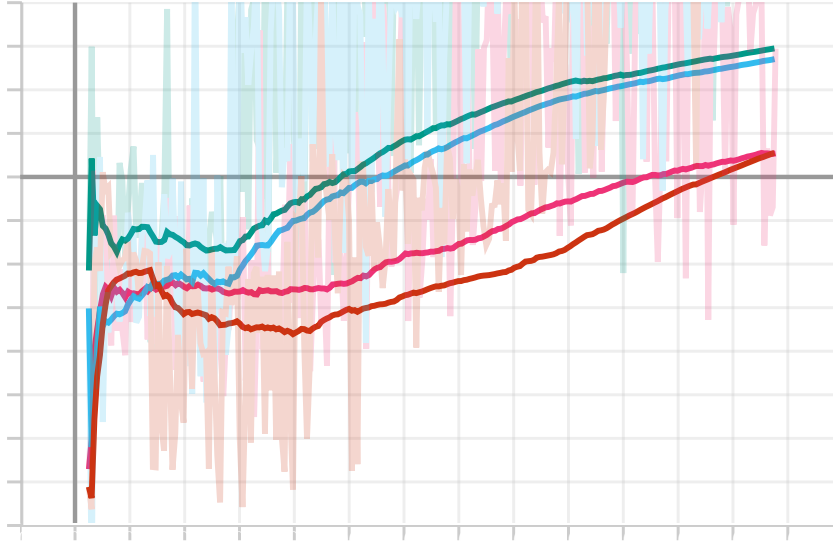


Figure 4: Valeur moyenne des rewards

Ici, les courbes verte et bleue représentent l'ajout de l'Action Wrapper. On remarque que leur évolution suit une tendance différente par rapport aux autres courbes, illustrant l'impact de cette modification sur les performances de l'agent.

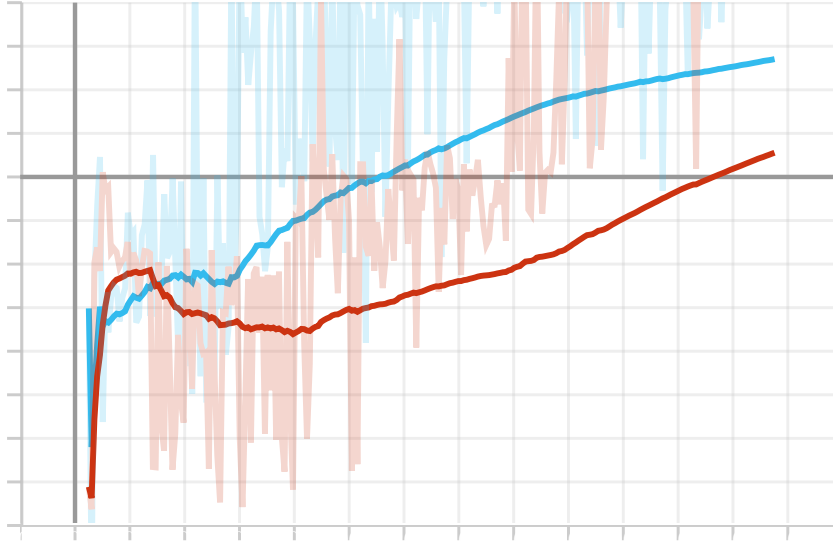


Figure 5: Valeur moyenne des rewards

Pour avoir une vision plus détaillée du phénomène, intéressons-nous à l’Action Wrapper XY. Au début, les performances ne sont pas exceptionnelles, mais au fur et à mesure que le temps passe, elles s’améliorent progressivement. Cette amélioration peut être attribuée à l’apprentissage progressif de l’agent, qui s’adapte de mieux en mieux à son environnement grâce à l’utilisation de l’Action Wrapper.

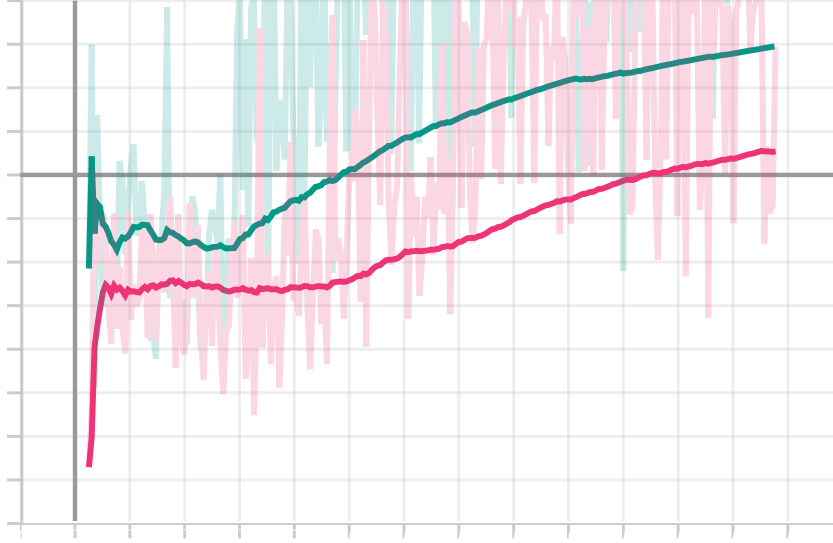


Figure 6: Valeur moyenne des rewards

Par contre, ici, dès le début, l’Action Wrapper offre de meilleures performances. On observe une progression plus rapide par rapport aux autres scénarios, ce qui suggère que son impact positif est également lié à la mémoire apportée par les positions (x, y) . Cette mémoire permet probablement à l’agent d’avoir une meilleure perception de son environnement et d’adapter plus efficacement ses actions.

3.3 Avec mémoire des positions (x,y) et sur θ

Pour finir, nous allons analyser l’impact de l’Action Wrapper sur l’environnement partiellement observé, défini avec l’intégration de toutes les mémoires. Cette configuration permet d’évaluer comment l’Action Wrapper interagit avec une meilleure rétention d’information et dans quelle mesure il influence les performances de l’agent dans un contexte où l’observation est limitée. Nous pourrions penser qu’il entraînera de meilleurs résultats, car l’ajout de mémoire combiné à l’Action Wrapper devrait théoriquement permettre une prise de décision plus efficace. Toutefois, il reste à vérifier si cette amélioration se traduit concrètement dans les performances observées.

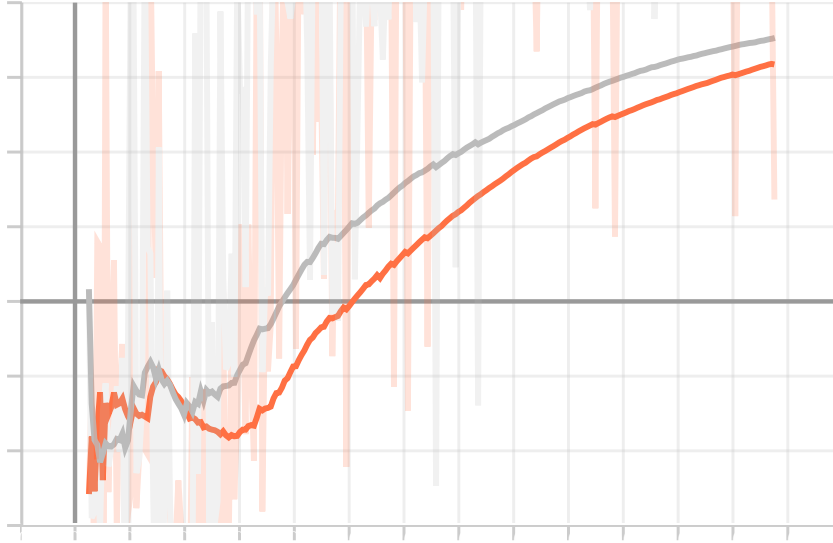


Figure 7: Valeur moyenne des rewards

Le graphique nous montre que l'ajout de l'Action Wrapper n'apporte pas une très grande plus-value. L'algorithme a du mal à s'adapter au début et finit par converger vers le même taux d'accroissement que sans Action Wrapper. Ainsi, nous pouvons conclure que l'Action Wrapper, lorsqu'il est combiné à un trop grand nombre de paramètres, peut perturber l'apprentissage et nuire aux performances de l'agent.

3.4 Conclusion

L'Action Wrapper peut être utile dans des environnements où l'agent dispose de très peu d'informations sur son état ou son environnement. En revanche, dans des contextes où l'agent possède déjà une grande quantité d'informations, son utilisation peut entraîner une instabilité dans l'apprentissage. De plus, chaque action prédite ajoute un coût computationnel exponentiel, rendant les calculs plus lourds. Il est donc essentiel d'effectuer un travail d'optimisation afin de trouver un équilibre entre la quantité d'informations fournies à l'agent et l'utilisation de l'Action Wrapper pour maximiser ses performances.